VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Yogaraj KH (1WA23CS054)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Jan-2025 to May-2026

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **Yogaraj KH (1WA23CS054), who is bonafide student of B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Amrutha B Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

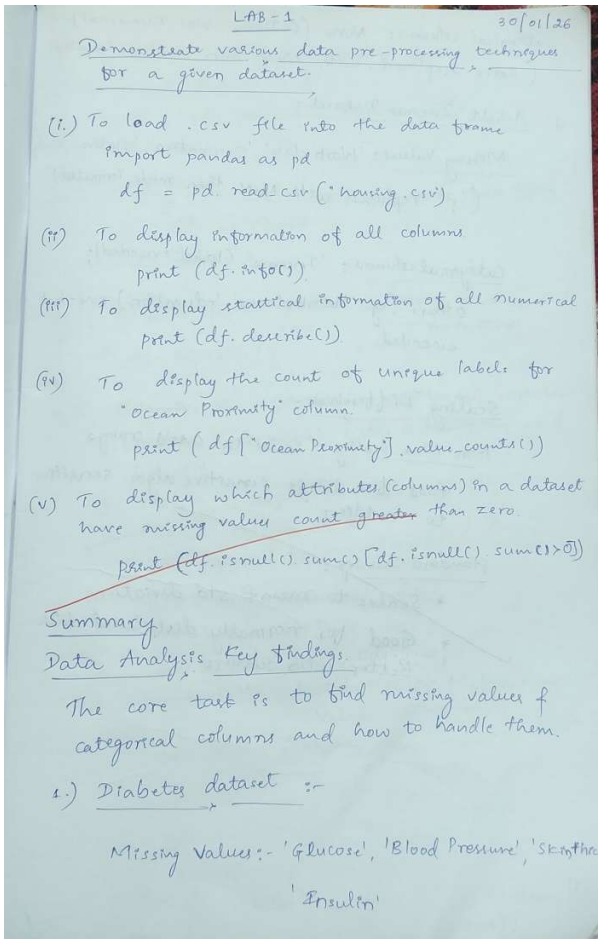
Index

Github Link: https://github.com/yogarajkh/ML-lab

Program 1

Write a python program to import and export data using Pandas library functions

Screenshot


LAB - 1 30/01/26

Demonstrate various data pre-processing techniques for a given dataset.

(i.) To load .csv file into the data frame
import pandas as pd
df = pd.read_csv("housing.csv")

(ii) To display information of all columns
print (df.info())

(iii) To display statical information of all numerical
print (df.describe())

(iv) To display the count of unique labels for "Ocean Proximity" column
print (df["Ocean Proximity"].value_counts())

(v) To display which attributes (columns) in a dataset have missing values count greater than zero
print (df.isnull().sum()[df.isnull().sum()>0])

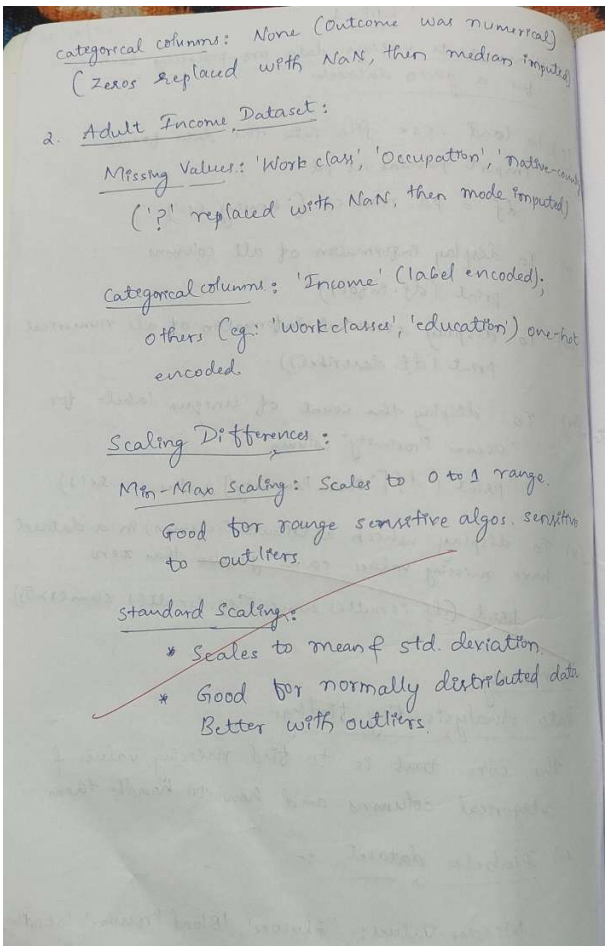
Summary

Data Analysis Key findings

The core task is to find missing values & categorical columns and how to handle them.

1.) Diabetes dataset :-

Missing Values :- 'Glucose', 'Blood Pressure', 'Skinthic
'Insulin'


categorical columns: None (Outcome was numerical)
(Zeros replaced with NaN, then median imputed)

2. Adult Income Dataset:

Missing Values: 'Work class', 'Occupation', 'native-country'
('?' replaced with NaN, then mode imputed)

Categorical columns: 'Income' (label encoded),
others (eg: 'workclasses', 'education') one-hot encoded.

Scaling Differences:

Min-Max scaling: Scales to 0 to 1 range.
Good for range sensitive algos. sensitive to outliers.

Standard Scaling:

* Scales to mean & std. deviation.
* Good for normally distributed data
Better with outliers.

Code:

```
import pandas as pd
import numpy as np

diabetes = pd.read_csv("diabetes.csv")
print("First 5 Records:")
print(diabetes.head())
print("\nDiabetes nulls per column:")
print(diabetes.isnull().sum())
print("\nDiabetes describe:")
print(diabetes.describe())
diabetes["BMI_Category"] = np.where(diabetes["BMI"] > 25, "High", "Normal")
print("\nUpdated Records:")
print(diabetes.head())
diabetes.to_csv("diabetes_output.csv", index=False)
print("\nData exported successfully to diabetes_output.csv")
adult = pd.read_csv("adult.csv")

print("\nAdult describe:")
print(adult.describe())

adult.replace("?", np.nan, inplace=True)
print("\nAdult nulls per column (after replacing '?' with NaN):")
print(adult.isnull().sum())
```

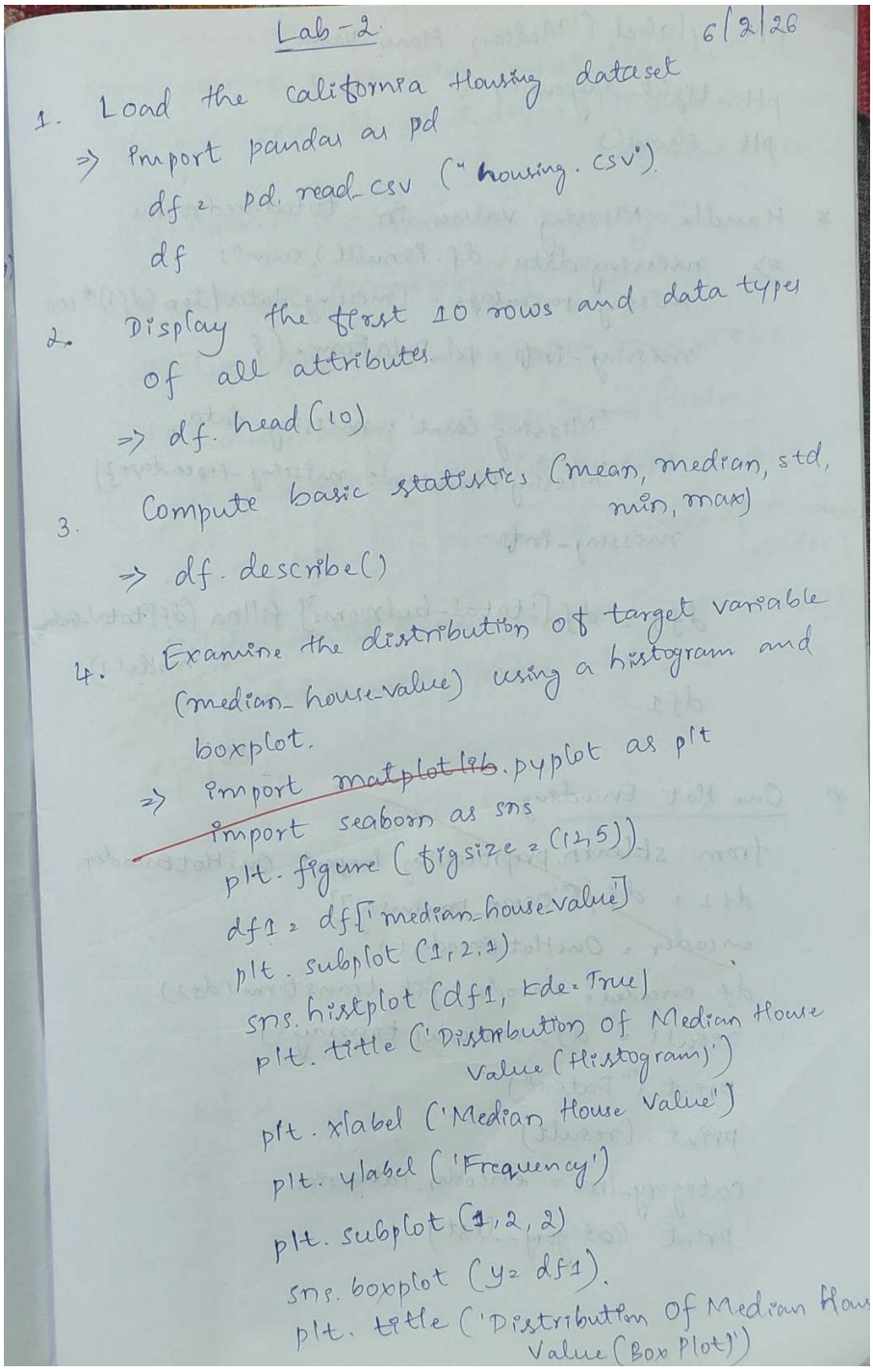
```
adult.dropna(inplace=True)

cat_cols = adult.select_dtypes(include="object").columns
print("\nAdult categorical columns:")
print(cat_cols)

le = LabelEncoder()
for col in cat_cols:
    adult[col] = le.fit_transform(adult[col])

num_cols = [
    "age", "fnlwgt", "educational-num",
    "capital-gain", "capital-loss", "hours-per-week"
]

Q1 = adult[num_cols].quantile(0.25)
Q3 = adult[num_cols].quantile(0.75)
IQR = Q3 - Q1

adult = adult[~(
    (adult[num_cols] < (Q1 - 1.5 * IQR)) |
    (adult[num_cols] > (Q3 + 1.5 * IQR))
).any(axis=1)]
```

```
scaler = MinMaxScaler()
adult[num_cols] = scaler.fit_transform(adult[num_cols])

scaler = StandardScaler()
adult[num_cols] = scaler.fit_transform(adult[num_cols])

print("\nAdult (after preprocessing) head:")
print(adult.head())
```

Program 2

Demonstrate various data pre-processing techniques for a given dataset.

Screenshot



Lab-2 6/2/26

1. Load the California Housing dataset

```
=> import pandas as pd
   df = pd.read_csv("housing.csv")
   df
```

2. Display the first 10 rows and data types of all attributes

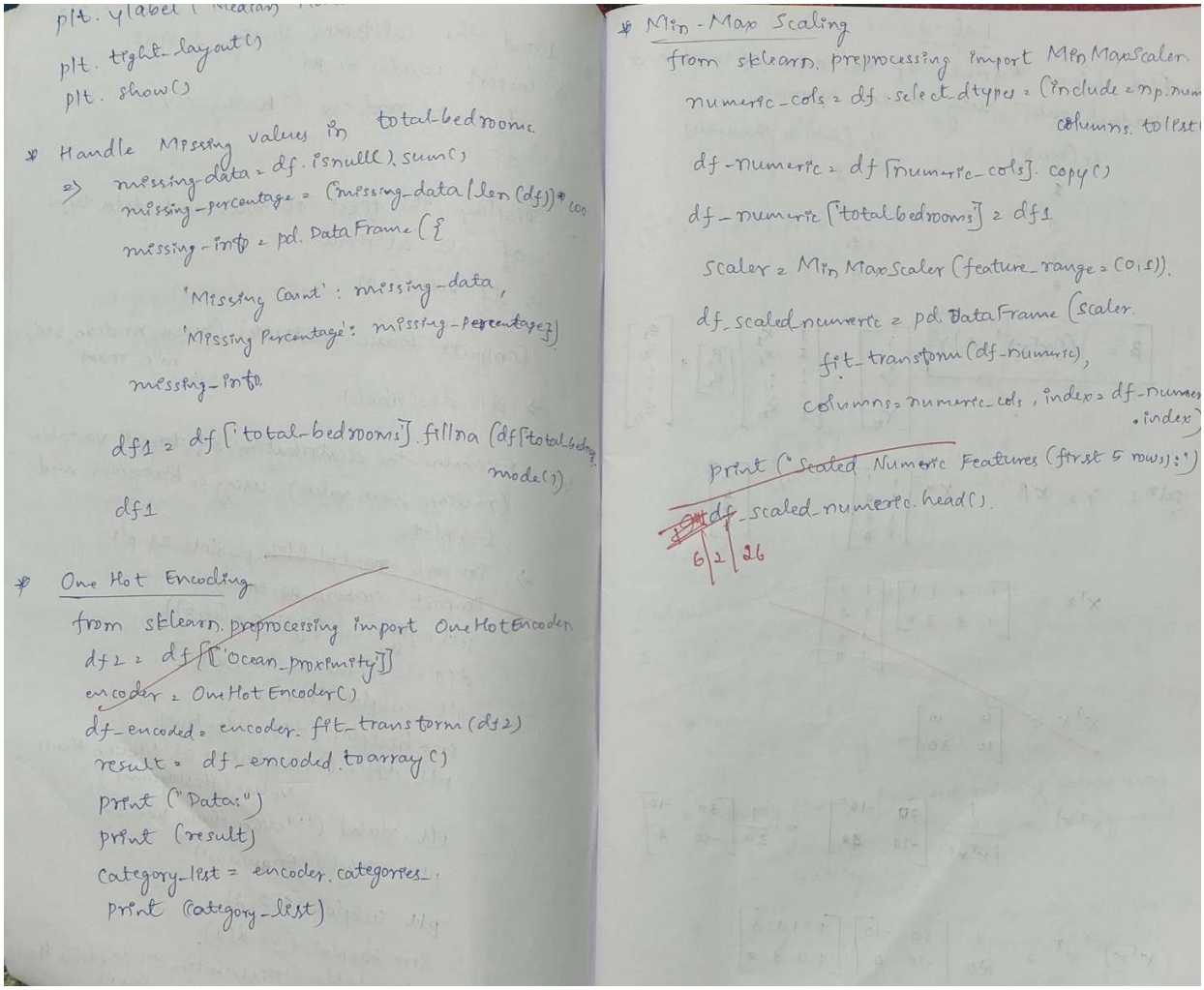
```
=> df.head(10)
```

3. Compute basic statistics (mean, median, std, min, max)

```
=> df.describe()
```

4. Examine the distribution of target variable (median_house_value) using a histogram and boxplot.

```
=> import matplotlib.pyplot as plt
   import seaborn as sns
   plt.figure(figsize=(12,5))
   df1 = df['median_house_value']
   plt.subplot(1,2,1)
   sns.histplot(df1, kde=True)
   plt.title('Distribution of Median House Value (Histogram)')
   plt.xlabel('Median House Value')
   plt.ylabel('Frequency')
   plt.subplot(1,2,2)
   sns.boxplot(y=df1)
   plt.title('Distribution of Median Hou
              Value (Box Plot)')
```



```
plt.ylabel("Median")
plt.tight_layout()
plt.show()

* Handle Missing values in total_bedrooms
=> missing_data = df.isnull().sum()
   missing_percentage = (missing_data / len(df)) * 100
   missing_info = pd.DataFrame({
       'Missing Count': missing_data,
       'Missing Percentage': missing_percentage})
   missing_info

   df1 = df['total_bedrooms'].fillna(df['total_bedrooms'].mode())
   df1

* One Hot Encoding
from sklearn.preprocessing import OneHotEncoder
df2 = df[['ocean_proximity']]
encoder = OneHotEncoder()
df_encoded = encoder.fit_transform(df2)
result = df_encoded.toarray()
print("Data:")
print(result)
category_list = encoder.categories_
print(category_list)

* Min-Max Scaling
from sklearn.preprocessing import MinMaxScaler
numeric_cols = df.select_dtypes(include=np.num
                                columns.tolist
df_numeric = df[numeric_cols].copy()
df_numeric['total_bedrooms'] = df1
scaler = MinMaxScaler(feature_range=(0,1))
df_scaled_numeric = pd.DataFrame(scaler.
        fit_transform(df_numeric),
        columns=numeric_cols, index=df_numer
                                .index)
print("Scaled Numeric Features (first 5 rows):")
df_scaled_numeric.head()
6 | 2 | 26
```

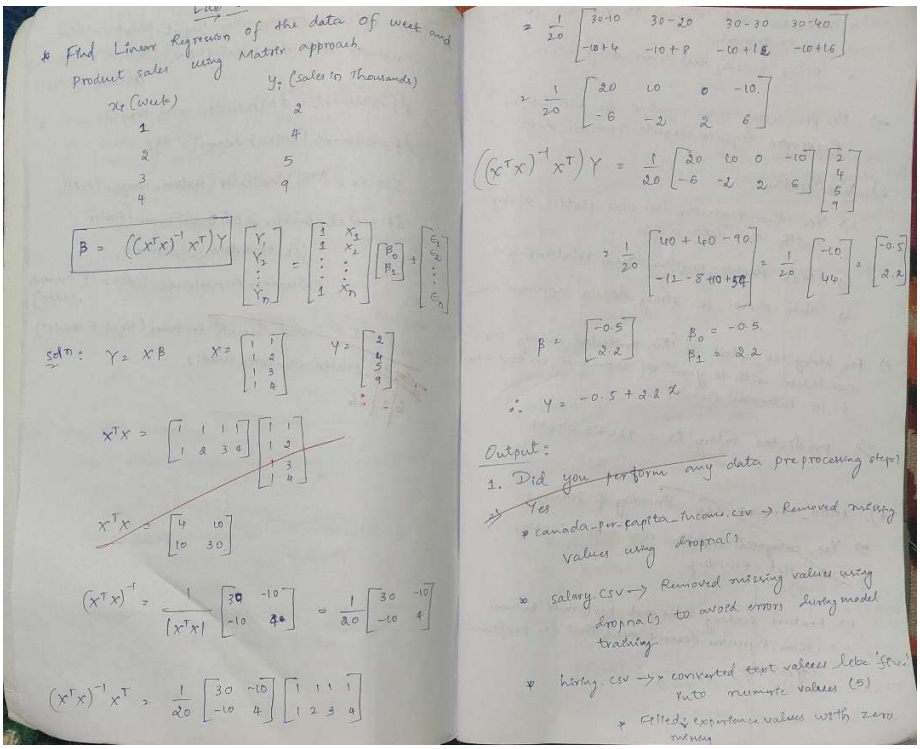
```
import pandas as pd

url = "https://raw.githubusercontent.com/ageron/handson-ml/master/datasets/housing/housing.csv"
housing = pd.read_csv(url)

housing.head(10)

housing.dtypes

housing.describe()

import matplotlib.pyplot as plt

plt.figure()
housing["median_house_value"].hist(bins=50)
plt.xlabel("Median House Value")
```

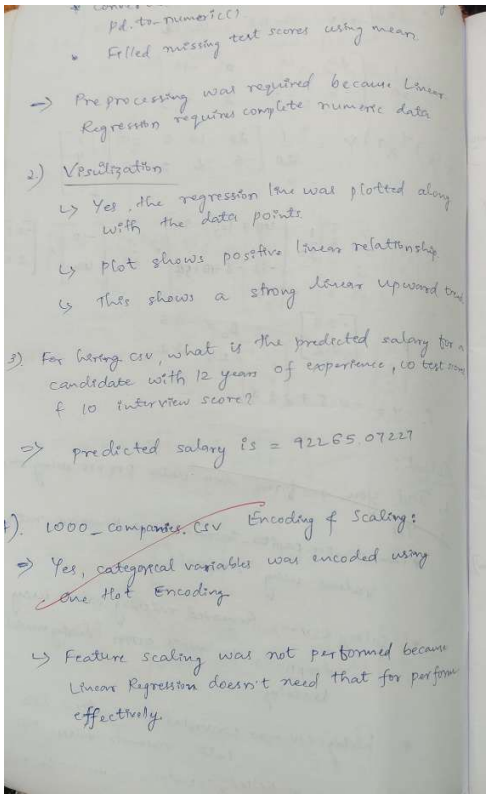
```
plt.ylabel("Frequency")
plt.title("Histogram of Median House Value")
plt.show()

plt.figure()
plt.boxplot(housing["median_house_value"], vert=False)
plt.xlabel("Median House Value")
plt.title("Box Plot of Median House Value")
plt.show()

missing_count = housing.isnull().sum()

missing_percentage = (missing_count / len(housing)) * 100

missing_data = pd.DataFrame({
   "Missing_Count": missing_count,
   "Missing_Percentage": missing_percentage
})

missing_data
```

Program 3

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Screenshot





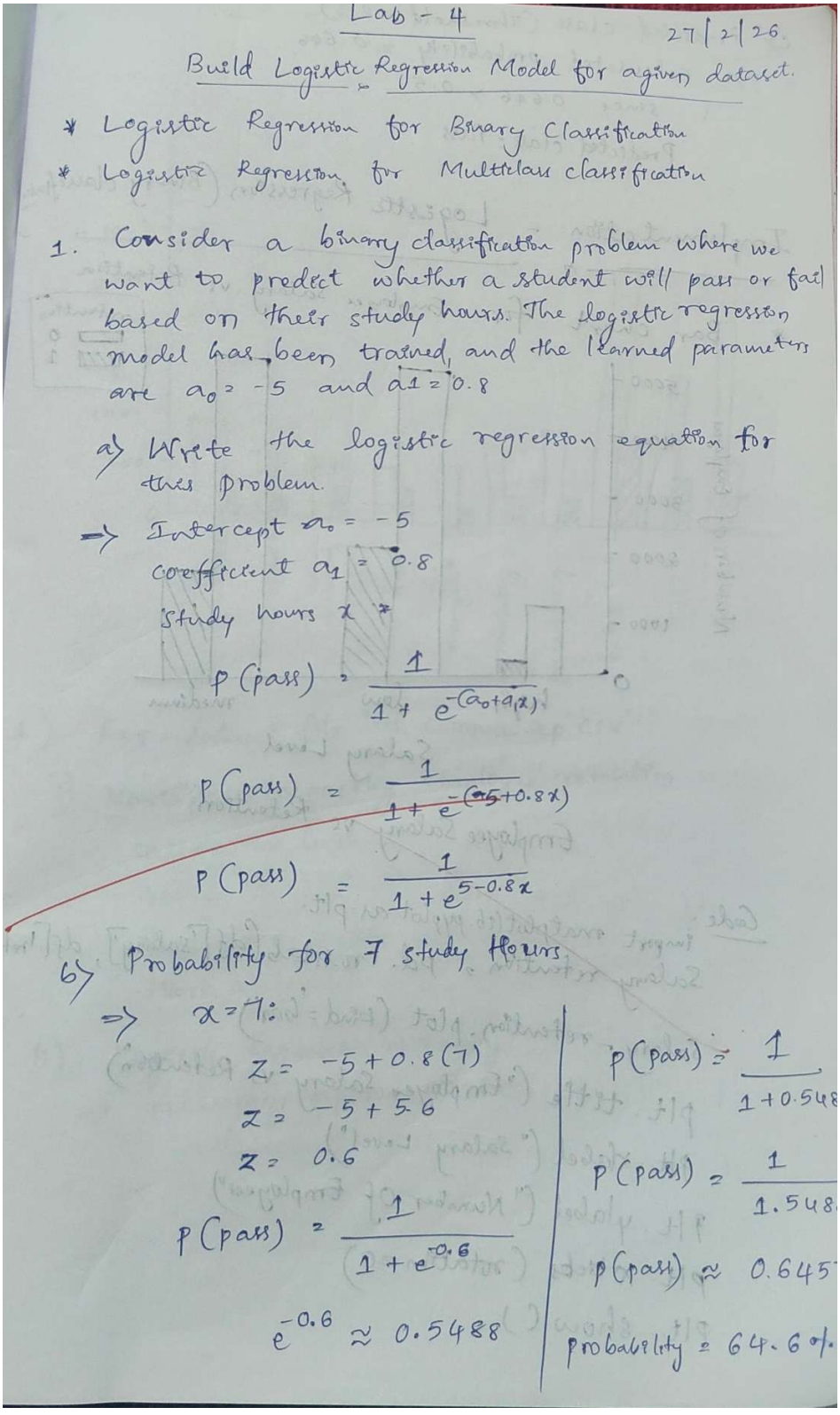
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

df_per_capita_income = pd.read_csv("canada_per_capita_income.csv")
df_per_capita_income.head()
print(df_per_capita_income.isnull().sum())

X = df_per_capita_income[['year']]
y = df_per_capita_income['per capita income (US$)']

model = LinearRegression()
model.fit(X, y)

prediction = model.predict([[2020]])
print("Predicted income in 2020:", prediction)

plt.scatter(X, y)
plt.plot(X, model.predict(X), color='red')
plt.show()

df_salary = pd.read_csv("salary.csv")
print(df_salary.isnull().sum())
df_salary = df_salary.dropna()
print(df_salary.isnull().sum())
print("Salary for 12 years:", model.predict([[12]]))

df_hiring = pd.read_csv("hiring.csv")
df_hiring.head()
print(df_hiring['experience'].unique())

word_to_num = {
    'zero': 0,
    'one': 1,
    'two': 2,
    'three': 3,
    'four': 4,
    'five': 5,
    'six': 6,
    'seven': 7,
    'eight': 8,
    'nine': 9,
    'ten': 10,
    'eleven': 11,
    'twelve': 12
}
df_hiring['experience'] = df_hiring['experience'].replace(word_to_num)
df_hiring['experience'] = pd.to_numeric(df_hiring['experience'])
df_hiring['experience'] = df_hiring['experience'].fillna(0)
print(df_hiring['experience'].unique())
print(df_hiring.isnull().sum())
```

```
df_hiring['test_score(out of 10)'] = df_hiring['test_score(out of 10)'].fillna(df_hiring['test_score(out of 10)'].median())

X = df_hiring[['experience', 'test_score(out of 10)', 'interview_score(out of 10)']]
y = df_hiring['salary($)']

model = LinearRegression()
model.fit(X, y)
print(model.predict([[2, 9, 6]]))
print(model.predict([[12, 10, 10]]))

df_companies = pd.read_csv("1000_Companies.csv")
df_companies.head()

df_companies = pd.get_dummies(
    df_companies,
    columns=["State"],
    drop_first=True
)
print(df_companies.head())

X = df_companies.drop("Profit", axis=1)
y = df_companies["Profit"]

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
print("R² Score:", model.score(X_test, y_test))

print(X.columns)
```

Program 4

Build Logistic Regression Model for a given dataset

Screenshot



Lab - 4

27/2/26

Build Logistic Regression Model for a given dataset.

* Logistic Regression for Binary Classification
* Logistic Regression for Multiclass classification

1. Consider a binary classification problem where we want to predict whether a student will pass or fail based on their study hours. The logistic regression model has been trained, and the learned parameters are $a_0 = -5$ and $a_1 = 0.8$

a) Write the logistic regression equation for this problem.

=> Intercept $a_0 = -5$

coefficient $a_1 = 0.8$

study hours x

$$P(\text{pass}) = \frac{1}{1 + e^{-(a_0 + a_1 x)}}$$

$$P(\text{pass}) = \frac{1}{1 + e^{-(-5 + 0.8x)}}$$

$$P(\text{pass}) = \frac{1}{1 + e^{5 - 0.8x}}$$

b) Probability for 7 study Hours

=> $x = 7$:

$$z = -5 + 0.8(7)$$

$$z = -5 + 5.6$$

$$z = 0.6$$

$$P(\text{pass}) = \frac{1}{1 + e^{-0.6}}$$

$$e^{-0.6} \approx 0.5488$$

$$P(\text{pass}) = \frac{1}{1 + 0.548}$$

$$P(\text{pass}) = \frac{1}{1.548}$$

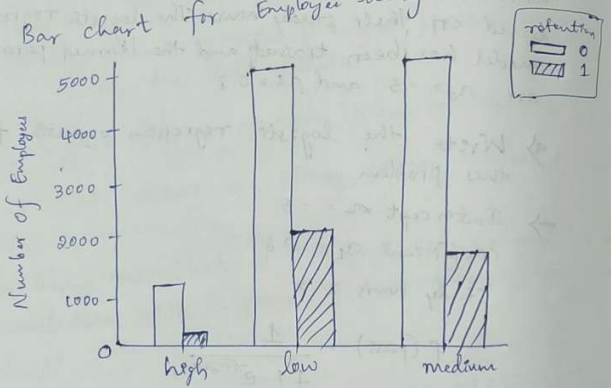
$$P(\text{pass}) \approx 0.645$$

probability = 64.6%

Predicted class (
=> Calculated probability = 0.646
since 0.646 > 0.5
Predicted class: Pass

Implementation - Logistic Regression (Binary classification)

* Bar chart for Employee Salary vs Retention

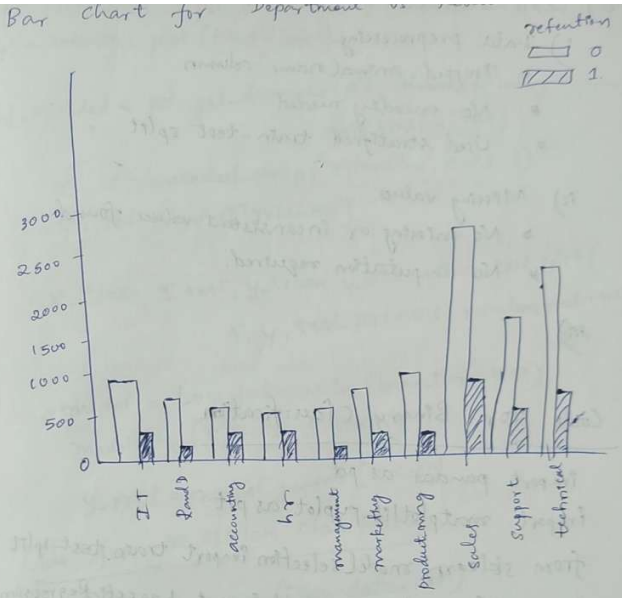


Employee Salary vs Retention.

Code:

```
import matplotlib.pyplot as plt.
Salary_retention = pd.crosstab(df['Salary'], df[retention])
Salary_retention.plot (kind='bar')
plt.title ("Employee Salary vs Retention")
plt.xlabel ("Salary Level")
plt.ylabel ("Number Of Employees")
plt.xticks (rotation=0)
plt.show()
```

* Bar chart for Department vs



1.) For dataset file "HR_comma_sep.csv"

i) Variables impacting employee retention
- satisfaction_level
- Salary
- time_spend_company
- Work_accident.

ii) Logistic Regression Accuracy
=> Accuracy approx 80-82%

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler

df = pd.read_csv('HR_comma_sep.csv')
df.head(5)
print(df.isnull().sum())

pd.crosstab(df.salary, df.left).plot(kind='bar')
plt.title("Salary vs Employee Retention")
plt.xlabel("Salary")
plt.ylabel("Count")
plt.show()

pd.crosstab(df.Department, df.left).plot(kind='bar', figsize=(10,5))
plt.title("Department vs Employee Retention")
plt.xlabel("Department")
plt.ylabel("Count")
```

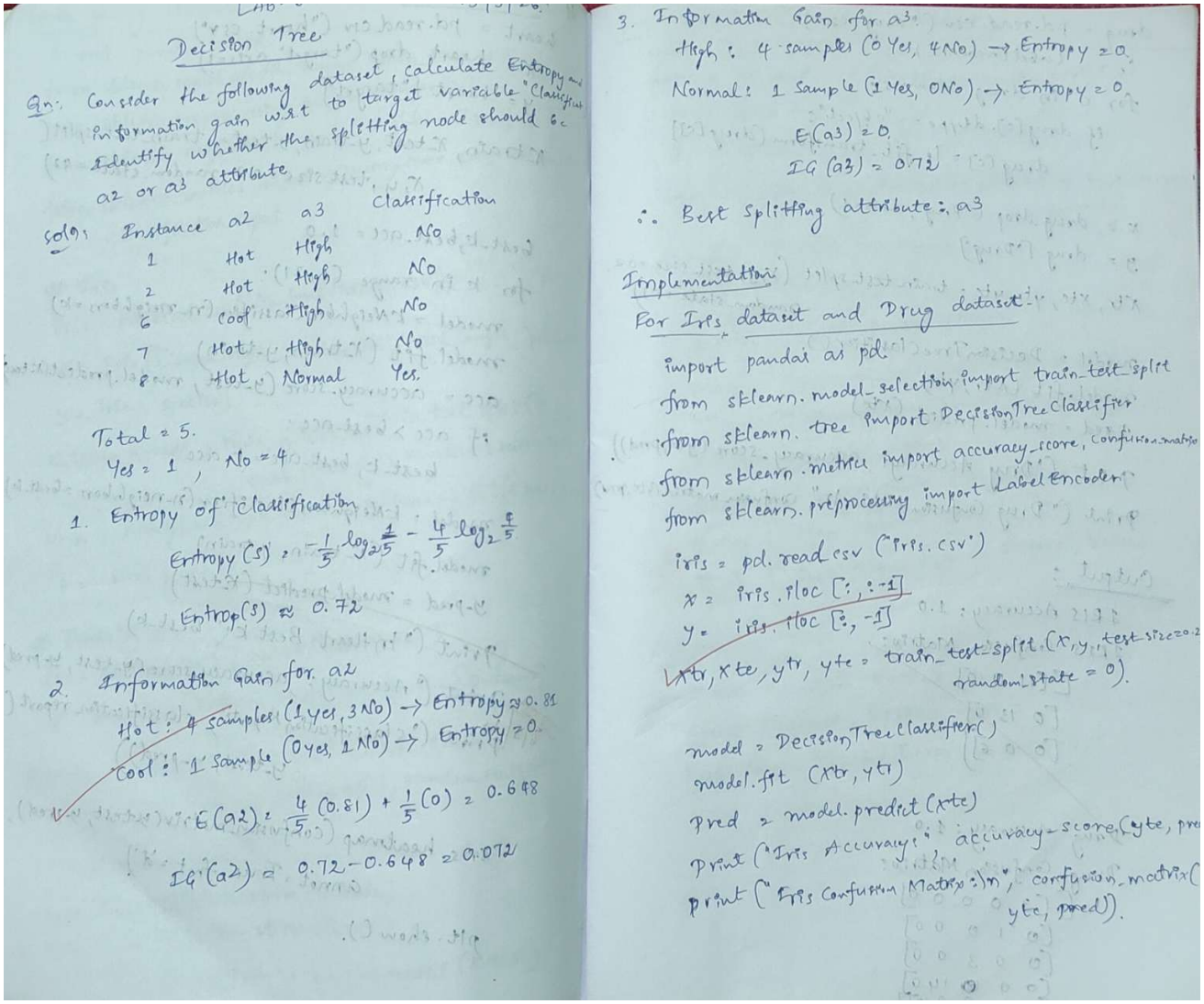
```
plt.show()

df = pd.get_dummies(df, columns=["salary", "Department"], drop_first=True)

X = df.drop("left", axis=1)
y = df["left"]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap="Blues")
plt.title("Confusion Matrix - HR Dataset")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler

zoo = pd.read_csv("zoo-data.csv")
```

Program 5

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample

Screenshot



Decision Tree

Qn: Consider the following dataset, calculate Entropy and information gain w.r.t to target variable "Classification". Identify whether the splitting node should be a2 or a3 attribute

Soln:

Instance	a2	a3	Classification
1	Hot	High	No
2	Hot	High	No
6	Cool	High	No
7	Hot	High	No
8	Hot	Normal	Yes

Total = 5.
Yes = 1, No = 4

1. Entropy of classification

$$\text{Entropy}(S) = -\frac{1}{5}\log_2\frac{1}{5} - \frac{4}{5}\log_2\frac{4}{5}$$

$$\text{Entropy}(S) \approx 0.72$$

2. Information Gain for a2

Hot: 4 samples (1 yes, 3 No) → Entropy ≈ 0.81
Cool: 1 sample (0 yes, 1 No) → Entropy = 0

$$E(a2) = \frac{4}{5}(0.81) + \frac{1}{5}(0) = 0.648$$

$$IG(a2) = 0.72 - 0.648 = 0.072$$

3. Information Gain for a3

High: 4 samples (0 Yes, 4 No) → Entropy = 0
Normal: 1 sample (1 Yes, 0 No) → Entropy = 0

$$E(a3) = 0$$

$$IG(a3) = 0.72$$

∴ Best splitting attribute: a3

Implementation:

For Iris dataset and Drug dataset:-

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.preprocessing import LabelEncoder
iris = pd.read_csv("iris.csv")
x = iris.iloc[:, :-1]
y = iris.iloc[:, -1]
xtr, xte, ytr, yte = train_test_split(x, y, test_size=0.2
                                      random_state = 0).

model = DecisionTreeClassifier()
model.fit(xtr, ytr)
pred = model.predict(xte)
print("Iris Accuracy:", accuracy_score(yte, pre
print("Iris Confusion Matrix:\n", confusion_matrix(
                                      yte, pred)).
```

```
drug = pd.read_csv("drug200.csv")
le = LabelEncoder()
for c in drug.columns:
    if drug[c].dtype == "object":
        drug[c] = le.fit_transform(drug[c])

x = drug.drop("Drug", axis=1)
y = drug["Drug"]
xtr, xte, ytr, yte = train_test_split(X, y, test_size=0.2,
                                      random_state=0))

model = DecisionTreeClassifier().
model.fit(xtr, ytr)
pred = model.predict(xte)
print("Drug Accuracy:", accuracy_score(yte, pred))
print("Drug Confusion Matrix:\n", confusion_matrix(yte, pred))
```

Output:

```
IRIS Accuracy: 1.0
IRIS Confusion Matrix:
[[11  0  0]
 [ 0 13  0]
 [ 0  0  6]]

Drug Accuracy: 1.0
Drug Confusion Matrix:
[[3 0 0 0 0]
 [0 1 0 0 0]
 [0 0 3 0 0]
```

for petrol consumption dataset

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
data = pd.read_csv("petrol_consumption.csv")
X = data.iloc[:, :-1]
y = data.iloc[:, -1]

xtr, xte, ytr, yte = train_test_split(X, y, test_size=0.2,
                                      random_state=0)).

model = DecisionTreeRegressor()
model.fit(xtr, ytr)

y_pred = model.predict(xte)
mae = mean_absolute_error(yte, y_pred)
mse = mean_squared_error(yte, y_pred)
rmse = np.sqrt(mse)
print("Mean Absolute Error:", mae)
print("Mean Squared Error:", mse)
print("Root Mean Squared Error:", rmse)
```

13/3/26

Output:

```
Mean Absolute Error: 52.3
Mean Squared Error: 4698.9
Root Mean Squared Error: 68.5485
```

```
import pandas as pd


drug = pd.read_csv("drug.csv")

iris = pd.read_csv("iris.csv")

petrol = pd.read_csv("petrol.csv")


drug.head()

iris.head()

petrol.head()


import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor

from sklearn.metrics import accuracy_score, confusion_matrix, mean_absolute_error,
mean_squared_error

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.tree import plot_tree


def plot_cm(cm, labels, title):

    plt.figure(figsize=(6, 5))

    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",

                xticklabels=labels, yticklabels=labels)

    plt.title(title)
```

```
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

X_iris = iris.drop("species", axis=1)
y_iris = iris["species"]

X_tr, X_te, y_tr, y_te = train_test_split(X_iris, y_iris,
                                          test_size=0.2, random_state=42)

clf_iris = DecisionTreeClassifier(random_state=42)
clf_iris.fit(X_tr, y_tr)
y_pred = clf_iris.predict(X_te)

print("=== IRIS ===")
print("Accuracy:", accuracy_score(y_te, y_pred))

cm_iris = confusion_matrix(y_te, y_pred)
plot_cm(cm_iris, clf_iris.classes_, "IRIS – Confusion Matrix")

plt.figure(figsize=(18, 8))
plot_tree(clf_iris, feature_names=X_iris.columns,
          class_names=clf_iris.classes_, filled=True, rounded=True)
```

```
plt.title("IRIS – Decision Tree")
plt.tight_layout()
plt.show()

plt.figure(figsize=(6, 4))
plt.barh(X_iris.columns, clf_iris.feature_importances_, color="steelblue")
plt.xlabel("Importance")
plt.title("IRIS – Feature Importances")
plt.tight_layout()
plt.show()

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, plot_tree
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             mean_absolute_error, mean_squared_error)
from sklearn.preprocessing import LabelEncoder

drug_enc = drug.copy()
le = LabelEncoder()
for col in ["Sex", "BP", "Cholesterol", "Drug"]:
    drug_enc[col] = le.fit_transform(drug_enc[col])

le_drug = LabelEncoder().fit(drug["Drug"])
```

```
X_drug = drug_enc.drop("Drug", axis=1)
y_drug = drug_enc["Drug"]

X_tr2, X_te2, y_tr2, y_te2 = train_test_split(X_drug, y_drug,
                                              test_size=0.2, random_state=42)

clf_drug = DecisionTreeClassifier(random_state=42)
clf_drug.fit(X_tr2, y_tr2)
y_pred2 = clf_drug.predict(X_te2)

print("\n=== DRUG ===")
print("Accuracy:", accuracy_score(y_te2, y_pred2))

cm_drug = confusion_matrix(y_te2, y_pred2)
plot_cm(cm_drug, le_drug.classes_, "DRUG – Confusion Matrix")

plt.figure(figsize=(6, 4))
plt.barh(X_drug.columns, clf_drug.feature_importances_, color="coral")
plt.xlabel("Importance")
plt.title("DRUG – Feature Importances")
plt.tight_layout()
plt.show()

X_pet = petrol.drop("Petrol_Consumption", axis=1)
```

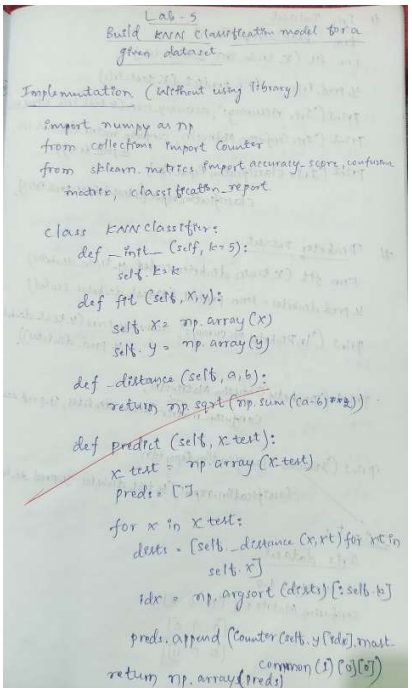
```
y_pet = petrol["Petrol_Consumption"]

X_tr3, X_te3, y_tr3, y_te3 = train_test_split(X_pet, y_pet,
                                    test_size=0.2, random_state=42)

reg = DecisionTreeRegressor(random_state=42)
reg.fit(X_tr3, y_tr3)
y_pred3 = reg.predict(X_te3)

mae = mean_absolute_error(y_te3, y_pred3)
mse = mean_squared_error(y_te3, y_pred3)
rmse = np.sqrt(mse)

print("\n=== PETROL CONSUMPTION (Regression) ===")
print(f"MAE : {mae:.2f}")
print(f"MSE : {mse:.2f}")
print(f"RMSE: {rmse:.2f}")

plt.figure(figsize=(6, 5))
plt.scatter(y_te3, y_pred3, color="teal", edgecolors="k", alpha=0.7)
plt.plot([y_te3.min(), y_te3.max()],
      [y_te3.min(), y_te3.max()], "r--", linewidth=2)
plt.xlabel("Actual")
plt.ylabel("Predicted")
```

```
plt.title("Petrol – Actual vs Predicted")
plt.tight_layout()
plt.show()

residuals = y_te3 - y_pred3
plt.figure(figsize=(6, 4))
plt.bar(range(len(residuals)), residuals, color="salmon")
plt.axhline(0, color="black", linewidth=1)
plt.xlabel("Test Sample Index")
plt.ylabel("Residual")
plt.title("Petrol – Residuals")
plt.tight_layout()
plt.show()

plt.figure(figsize=(6, 4))
plt.barh(X_pet.columns, reg.feature_importances_, color="mediumseagreen")
plt.xlabel("Importance")
plt.title("Petrol – Feature Importances")
plt.tight_layout()
plt.show()
```

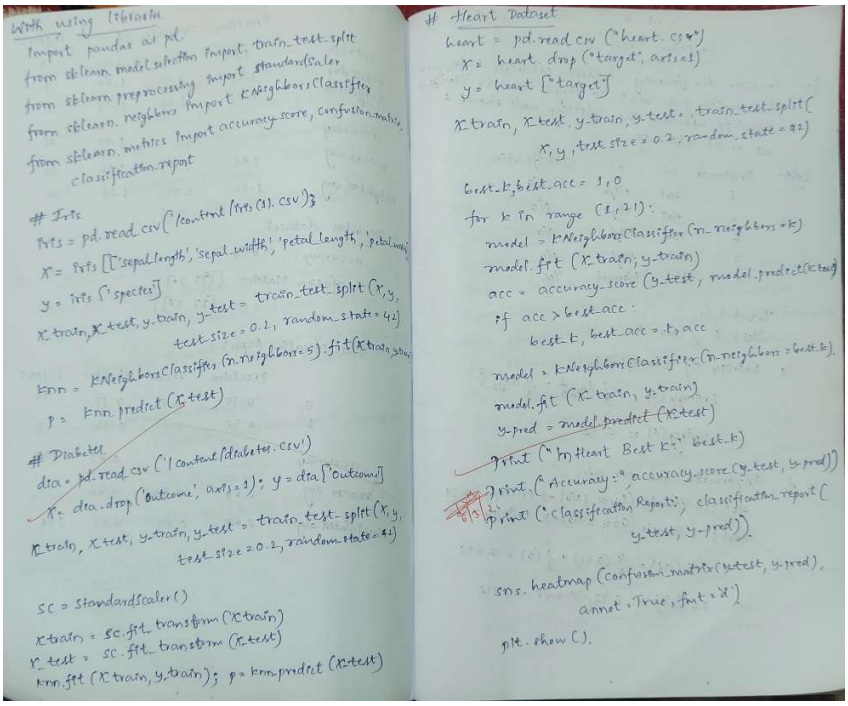
Program 6

Build KNN Classification model for a given dataset.

Screenshot



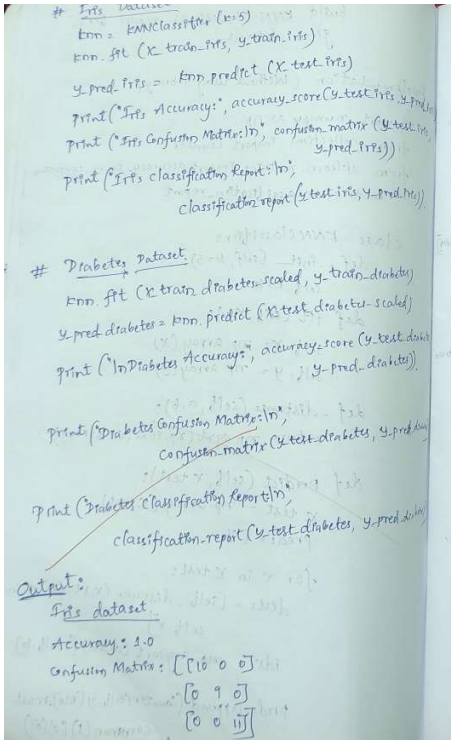
```
Lab - 5
Build KNN classification model for a
given dataset

Implementation (without using library)

import numpy as np
from collections import Counter
from sklearn.metrics import accuracy_score, confusion
matrix, classification_report

class KNNClassifier:
    def __init__(self, k=5):
        self.k = k
    def fit(self, X, y):
        self.X = np.array(X)
        self.y = np.array(y)
    def _distance(self, a, b):
        return np.sqrt(np.sum((a-b)**2))
    def predict(self, X_test):
        X_test = np.array(X_test)
        preds = []
        for x in X_test:
            dists = [self._distance(x, xt) for xt in
                     self.X]
            idx = np.argsort(dists)[:self.k]
            preds.append(Counter(self.y[idx]).most_
common(1)[0][0])
        return np.array(preds)
```



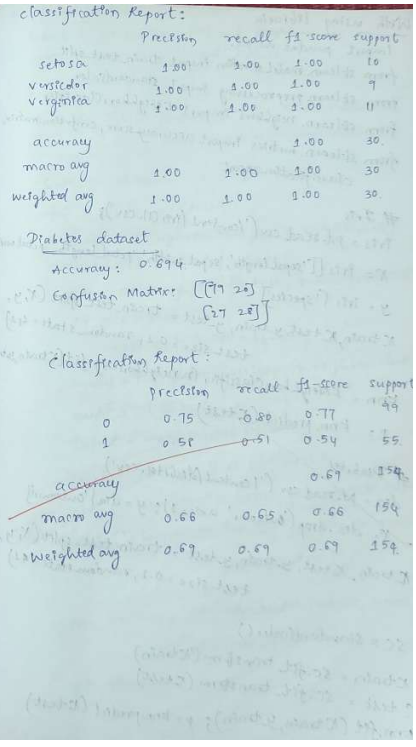
```
With using library

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
    classification_report

# Iris
iris = pd.read_csv('/content/Iris (1).csv');
X = iris[['sepal_length', 'sepal_width', 'petal_length', 'petal_wi
y = iris['species']
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)
knn = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_tra
p = knn.predict(X_test)

# Diabetes
dia = pd.read_csv('/content/diabetes.csv')
X = dia.drop('Outcome', axis=1); y = dia['Outcome']
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.fit_transform(X_test)
knn.fit(X_train, y_train); p = knn.predict(X_test)
```



```
# Heart Dataset
heart = pd.read_csv("heart.csv")
X = heart.drop("target", axis=1)
y = heart["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

best_k, best_acc = 1, 0
for k in range(1, 21):
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    acc = accuracy_score(y_test, model.predict(X_test))
    if acc > best_acc:
        best_k, best_acc = k, acc

model = KNeighborsClassifier(n_neighbors=best_k)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("\nHeart Best k:", best_k)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:", classification_report(
    y_test, y_pred))

sns.heatmap(confusion_matrix(y_test, y_pred),
    annot=True, fmt='d')
plt.show().
```



```
# Iris Dataset
knn = KNNClassifier(k=5)
knn.fit(X_train_iris, y_train_iris)
y_pred_iris = knn.predict(X_test_iris)
print("Iris Accuracy:", accuracy_score(y_test_iris, y_pred_
print("Iris Confusion Matrix:\n", confusion_matrix(y_test_iris,
    y_pred_iris))
print("Iris classification Report:\n",
    classification_report(y_test_iris, y_pred_iris))

# Diabetes Dataset
knn.fit(X_train_diabetes_scaled, y_train_diabetes)
y_pred_diabetes = knn.predict(X_test_diabetes_scaled)
print("\nDiabetes Accuracy:", accuracy_score(y_test_diabe
    y_pred_diabetes))

print("Diabetes Confusion Matrix:\n",
    confusion_matrix(y_test_diabetes, y_pred_diab
print("Diabetes Classification Report:\n",
    classification_report(y_test_diabetes, y_pred_diab

Output:
Iris dataset
Accuracy: 1.0
Confusion Matrix: [[10  0  0]
                   [ 0  9  0]
                   [ 0  0 11]]
```



```
Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30

Diabetes dataset
Accuracy: 0.694
Confusion Matrix: [[97 20]
                   [27 28]]

Classification Report:
              precision    recall  f1-score   support

           0       0.75      0.80      0.77        49
           1       0.58      0.51      0.54        55

    accuracy                           0.69       154
   macro avg       0.66      0.65      0.66       154
weighted avg       0.69      0.69      0.69       154
```

```
import pandas as pd

df = pd.read_csv('/content/iris (1).csv')

print("First 5 rows of the dataset:")
print(df.head())

print("\nColumn names:")
print(df.columns.tolist())

X = df.drop('species', axis=1)
y = df['species']

print("Features (X) shape:", X.shape)
print("Target (y) shape:", y.shape)
print("First 5 rows of X:")
print(X.head())
print("\nFirst 5 values of y:")
print(y.head())

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

```
print("X_train shape:", X_train.shape)

print("X_test shape:", X_test.shape)

print("y_train shape:", y_train.shape)

print("y_test shape:", y_test.shape)


from sklearn.neighbors import KNeighborsClassifier


knn_model = KNeighborsClassifier(n_neighbors=5)


print("KNN model initialized:")

print(knn_model)


knn_model.fit(X_train, y_train)


print("KNN model trained successfully.")


y_pred = knn_model.predict(X_test)


print("First 5 predictions on the test set:")

print(y_pred[:5])

print("Length of predictions:", len(y_pred))


from sklearn.metrics import accuracy_score
```

```
accuracy = accuracy_score(y_test, y_pred)

print(f"Model Accuracy: {accuracy:.4f}")

from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
        xticklabels=knn_model.classes_, yticklabels=knn_model.classes_)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

print("Confusion Matrix:\n", cm)

from sklearn.metrics import classification_report

report = classification_report(y_test, y_pred)
```

```
print("Classification Report:")
print(report)

import numpy as np

def euclidean_distance(point1, point2):
    """Calculates the Euclidean distance between two points."""
    return np.sqrt(np.sum((np.array(point1) - np.array(point2))**2))

print("Euclidean distance function defined.")

from collections import Counter

def predict_one(test_point, X_train, y_train, k):
    """Predicts the class of a single test point using KNN."""
    distances = []
    for i, train_point in enumerate(X_train.values):
        dist = euclidean_distance(test_point, train_point)
        distances.append((dist, y_train.iloc[i]))

    distances.sort(key=lambda x: x[0])

    k_nearest_neighbors = distances[:k]
    k_nearest_labels = [label for dist, label in k_nearest_neighbors]
```

```
    most_common = Counter(k_nearest_labels).most_common(1)
    return most_common[0][0]

print("predict_one function defined.")

def custom_knn_predict(X_test, X_train, y_train, k):
    """Makes predictions for the entire test set using the custom KNN algorithm."""
    predictions = []
    for i, test_point in enumerate(X_test.values):
        prediction = predict_one(test_point, X_train, y_train, k)
        predictions.append(prediction)
    return predictions

print("custom_knn_predict function defined.")

k = 5
y_pred_custom = custom_knn_predict(X_test, X_train, y_train, k)

print(f"First 5 custom KNN predictions (k={k}):")
print(y_pred_custom[:5])
print(f"Length of custom predictions: {len(y_pred_custom)}")

from sklearn.metrics import accuracy_score
```

```
accuracy_custom = accuracy_score(y_test, y_pred_custom)

print(f"Custom KNN Model Accuracy: {accuracy_custom:.4f}")

from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

class_labels = sorted(list(set(y_test).union(set(y_pred_custom))))

cm_custom = confusion_matrix(y_test, y_pred_custom, labels=class_labels)

plt.figure(figsize=(8, 6))
sns.heatmap(cm_custom, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_labels, yticklabels=class_labels)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Custom KNN Confusion Matrix')
plt.show()

print("Custom KNN Confusion Matrix:\n", cm_custom)

from sklearn.metrics import classification_report
```

```
report_custom = classification_report(y_test, y_pred_custom)

print("Custom KNN Classification Report:")
print(report_custom)

import pandas as pd

df_diabetes = pd.read_csv('/content/diabetes.csv')

print("First 5 rows of the diabetes dataset:")
print(df_diabetes.head())

print("\nColumn names of the diabetes dataset:")
print(df_diabetes.columns.tolist())

X_diabetes = df_diabetes.drop('Outcome', axis=1)
y_diabetes = df_diabetes['Outcome']

print("Features (X_diabetes) shape:", X_diabetes.shape)
print("Target (y_diabetes) shape:", y_diabetes.shape)
print("\nFirst 5 rows of X_diabetes:")
print(X_diabetes.head())
print("\nFirst 5 values of y_diabetes:")
```

```
print(y_diabetes.head())

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled_diabetes = scaler.fit_transform(X_diabetes)

X_scaled_diabetes = pd.DataFrame(X_scaled_diabetes, columns=X_diabetes.columns)

print("First 5 rows of scaled features (X_scaled_diabetes):")
print(X_scaled_diabetes.head())

from sklearn.model_selection import train_test_split

X_train_diabetes_scaled, X_test_diabetes_scaled, y_train_diabetes, y_test_diabetes =
train_test_split(
    X_scaled_diabetes, y_diabetes, test_size=0.2, random_state=42, stratify=y_diabetes
)

print("X_train_diabetes_scaled shape:", X_train_diabetes_scaled.shape)
print("X_test_diabetes_scaled shape:", X_test_diabetes_scaled.shape)
print("y_train_diabetes shape:", y_train_diabetes.shape)
print("y_test_diabetes shape:", y_test_diabetes.shape)
```

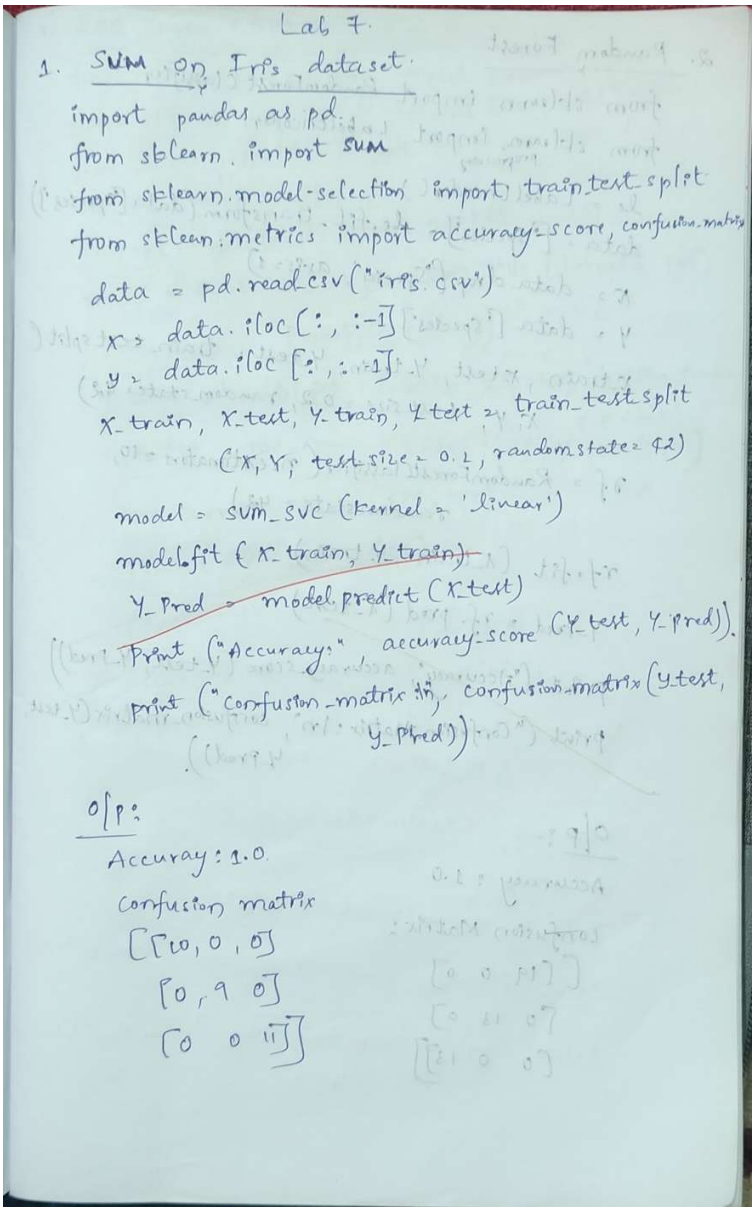
```
from sklearn.neighbors import KNeighborsClassifier

knn_model_diabetes = KNeighborsClassifier(n_neighbors=5)

print("KNN model for diabetes dataset initialized:")
print(knn_model_diabetes)

knn_model_diabetes.fit(X_train_diabetes_scaled, y_train_diabetes)

print("KNN model for diabetes dataset trained successfully.")

y_pred_diabetes = knn_model_diabetes.predict(X_test_diabetes_scaled)

print("First 5 predictions on the diabetes test set:")
print(y_pred_diabetes[:5])
print(f"Length of diabetes predictions: {len(y_pred_diabetes)}")

from sklearn.metrics import accuracy_score

accuracy_diabetes = accuracy_score(y_test_diabetes, y_pred_diabetes)

print(f"Diabetes KNN Model Accuracy: {accuracy_diabetes:.4f}")

from sklearn.metrics import confusion_matrix
```

```
import seaborn as sns
import matplotlib.pyplot as plt

class_labels_diabetes = sorted(list(set(y_test_diabetes).union(set(y_pred_diabetes))))

cm_diabetes = confusion_matrix(y_test_diabetes, y_pred_diabetes, labels=class_labels_diabetes)

plt.figure(figsize=(8, 6))
sns.heatmap(cm_diabetes, annot=True, fmt='d', cmap='Blues',
        xticklabels=class_labels_diabetes, yticklabels=class_labels_diabetes)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Diabetes KNN Confusion Matrix')
plt.show()

print("Diabetes KNN Confusion Matrix:\n", cm_diabetes)

from sklearn.metrics import classification_report

report_diabetes = classification_report(y_test_diabetes, y_pred_diabetes)

print("Diabetes KNN Classification Report:")
print(report_diabetes)
```

Program 7

Build Support vector machine model for a given dataset

Screenshot



Lab 7.

1. SVM on Iris dataset.

```
import pandas as pd
from sklearn import svm
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix
data = pd.read_csv("iris.csv")
x = data.iloc[:, :-1]
y = data.iloc[:, :-1]
X_train, X_test, Y_train, Y_test = train_test_split
        (X, Y, test_size = 0.2, random_state = 42)
model = svm.SVC(kernel = 'linear')
model.fit(X_train, Y_train)
Y_Pred = model.predict(X_test)
Print("Accuracy:", accuracy_score(Y_test, Y_pred))
print("confusion_matrix is", confusion_matrix(y_test,
        y_pred))
```

o/p:

```
Accuray: 1.0
confusion matrix
[[10, 0, 0]
 [0, 9  0]
 [0  0  11]]
```

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, confusion_matrix, roc_curve, auc,
ConfusionMatrixDisplay

from sklearn.preprocessing import label_binarize

import matplotlib.pyplot as plt


iris_df = pd.read_csv("iris.csv")


X_iris = iris_df.iloc[:, :-1]

y_iris = iris_df.iloc[:, -1]


X_train_i, X_test_i, y_train_i, y_test_i = train_test_split(

        X_iris, y_iris, test_size=0.2, random_state=42

)


svm_linear = SVC(kernel='linear')

svm_linear.fit(X_train_i, y_train_i)


y_pred_linear = svm_linear.predict(X_test_i)

acc_linear = accuracy_score(y_test_i, y_pred_linear)

cm_linear = confusion_matrix(y_test_i, y_pred_linear)
```

```
plt.figure()

disp = ConfusionMatrixDisplay(cm_linear)
disp.plot(cmap="Blues", values_format="d")

plt.title(f"IRIS Linear SVM\nAccuracy = {acc_linear:.4f}")
plt.grid(False)
plt.xticks(rotation=45)
plt.yticks(rotation=0)

plt.show()

svm_rbf = SVC(kernel='rbf')
svm_rbf.fit(X_train_i, y_train_i)

y_pred_rbf = svm_rbf.predict(X_test_i)
acc_rbf = accuracy_score(y_test_i, y_pred_rbf)
cm_rbf = confusion_matrix(y_test_i, y_pred_rbf)

plt.figure()

disp = ConfusionMatrixDisplay(cm_rbf)
disp.plot(cmap="Blues", values_format="d")
```

```
plt.title(f"IRIS RBF SVM\nAccuracy = {acc_rbf:.4f}")
plt.grid(False)
plt.xticks(rotation=45)
plt.yticks(rotation=0)

plt.show()

letter_df = pd.read_csv("letter-recognition.csv")

X_letter = letter_df.iloc[:, 1:]
y_letter = letter_df.iloc[:, 0]

X_train_l, X_test_l, y_train_l, y_test_l = train_test_split(
    X_letter, y_letter, test_size=0.2, random_state=42
)

svm_letter = SVC(kernel='rbf', probability=True)
svm_letter.fit(X_train_l, y_train_l)

y_pred_l = svm_letter.predict(X_test_l)
acc_letter = accuracy_score(y_test_l, y_pred_l)
cm_letter = confusion_matrix(y_test_l, y_pred_l)

plt.figure()
```

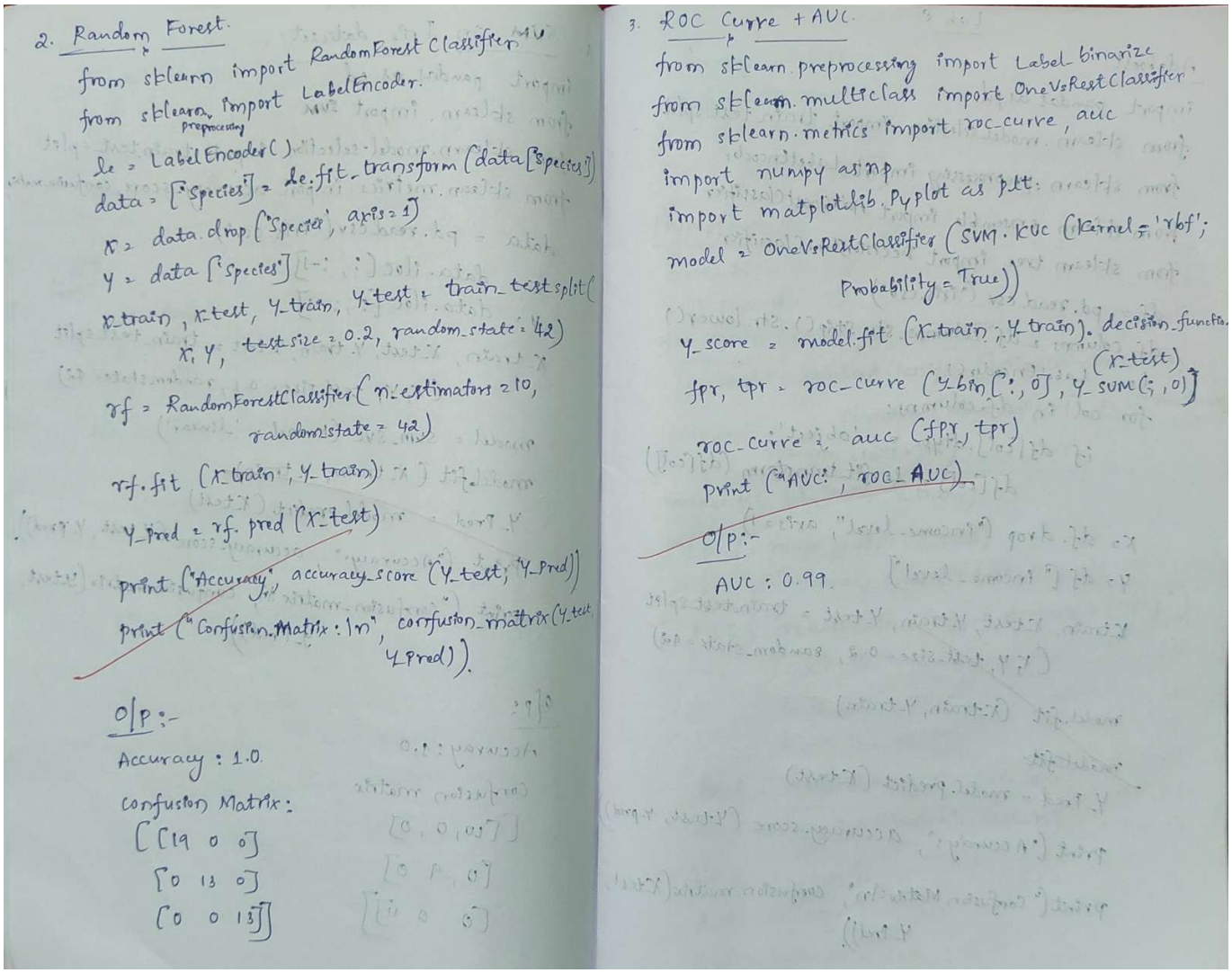
```
disp = ConfusionMatrixDisplay(cm_letter)
disp.plot(cmap="Blues", values_format="d")

plt.title(f"LETTER SVM Confusion Matrix\nAccuracy = {acc_letter:.4f}")
plt.grid(False)
plt.xticks(rotation=45)
plt.yticks(rotation=0)

plt.show()

classes = np.unique(y_letter)
y_test_bin = label_binarize(y_test_l, classes=classes)

y_score = svm_letter.predict_proba(X_test_l)

fpr = {}
tpr = {}
roc_auc = {}

for i in range(len(classes)):
    fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])
```

```
plt.figure()


for i in range(len(classes)):
    plt.plot(fpr[i], tpr[i], label=f"Class {classes[i]} (AUC = {roc_auc[i]:.2f})")


plt.plot([0, 1], [0, 1], linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Letter Recognition")
plt.legend()
plt.show()


print("Average AUC:", np.mean(list(roc_auc.values())))
```

Program 8

Implement Random forest ensemble method on a given dataset

Screenshot



2. Random Forest.

```
from sklearn import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder.
le = LabelEncoder()
data['Species'] = le.fit_transform(data['Species'])
X = data.drop('Species', axis=1)
y = data['Species']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size = 0.2, random_state = 42)
rf = RandomForestClassifier(n_estimators = 10,
                            random_state = 42)
rf.fit(X_train, y_train)
y_pred = rf.pred(X_test)
print("Accuracy", accuracy_score(y_test, y_pred))
print("Confusion Matrix : \n", confusion_matrix(y_test,
                                                y_pred)).
```

O/P :-

```
Accuracy : 1.0
Confusion Matrix :
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

3. ROC Curve + AUC.

```
from sklearn.preprocessing import Label_binarize
from sklearn.multiclass import OneVsRestClassifier
from sklearn.metrics import roc_curve, auc
import numpy as np
import matplotlib.pyplot as plt.
model = OneVsRestClassifier(SVM.SVC(kernel='rbf',
                            probability=True))
y_score = model.fit(X_train, y_train).decision_functio
fpr, tpr = roc_curve(y_bin[:, 0], y_sum(X_test)[:, 0])
roc_curve = auc(fpr, tpr)
print("AUC:", roc_AUC)
```

O/P :-

```
AUC : 0.99
```

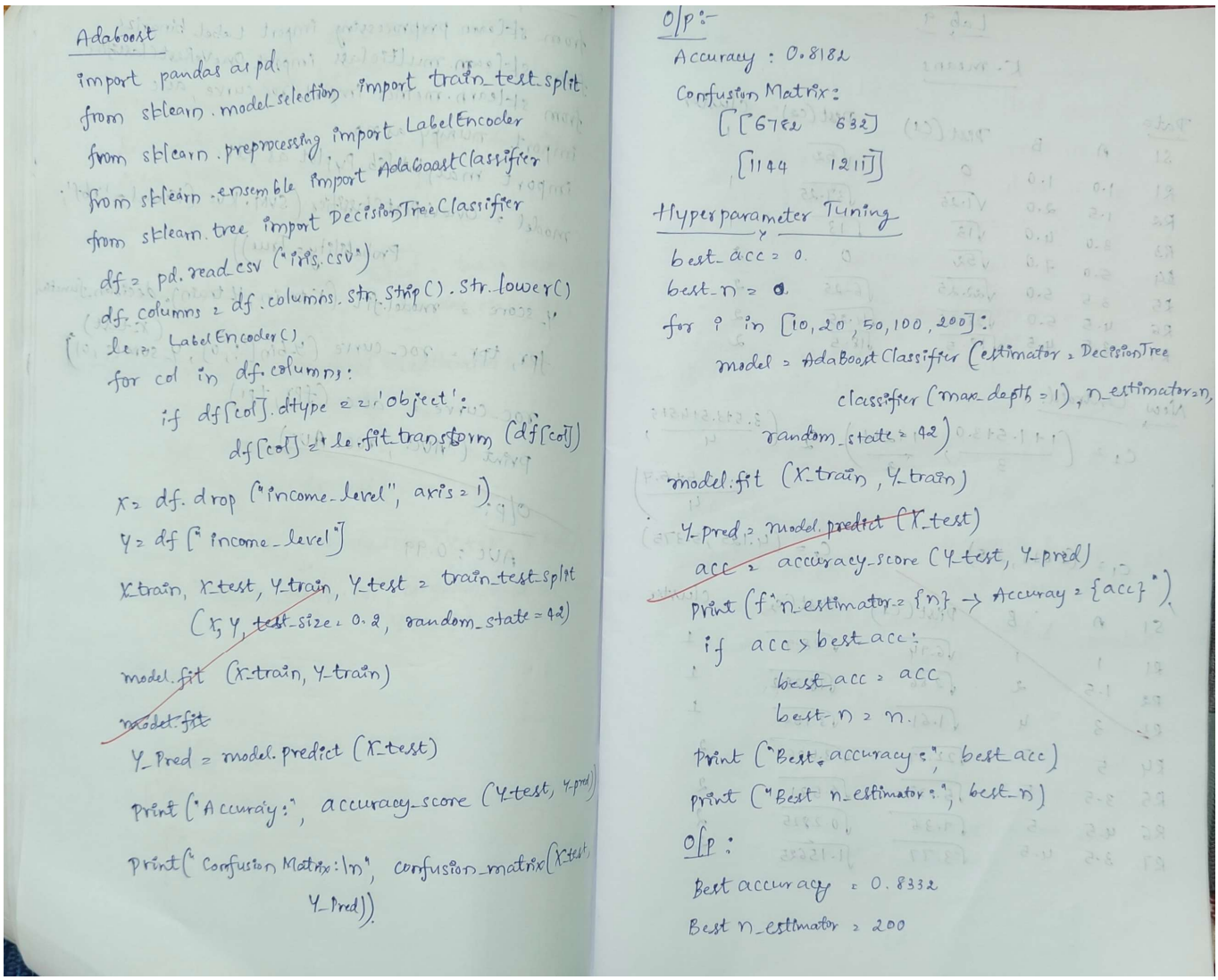
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

iris_df = pd.read_csv("iris.csv")

X = iris_df.iloc[:, :-1]
y = iris_df.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

rf_default = RandomForestClassifier(n_estimators=10, random_state=42)
rf_default.fit(X_train, y_train)

y_pred_default = rf_default.predict(X_test)
default_acc = accuracy_score(y_test, y_pred_default)

print("Default n_estimators = 10")
print("Accuracy:", default_acc)
```

```
tree_range = range(1, 101)
scores = []

for n in tree_range:
    rf = RandomForestClassifier(n_estimators=n, random_state=42)
    rf.fit(X_train, y_train)
    y_pred = rf.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    scores.append(acc)

best_score = max(scores)
best_trees = tree_range[scores.index(best_score)]

print("\nBest Accuracy:", best_score)
print("Optimal number of trees:", best_trees)

plt.figure()
plt.plot(tree_range, scores)
plt.xlabel("Number of Trees")
plt.ylabel("Accuracy")
plt.title("Random Forest Accuracy vs Number of Trees")
plt.show()
```

Program 9

Implement Boosting ensemble method on a given dataset.

Screenshot



```
Adaboost
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
df = pd.read_csv("iris.csv")
df.columns = df.columns.str.strip().str.lower()
le = LabelEncoder()
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
X = df.drop("income_level", axis=1)
y = df["income_level"]
X_train, X_test, y_train, y_test = train_test_split
    (X, y, test_size=0.2, random_state=42)
model.fit(X_train, y_train)
model.fit
y_Pred = model.predict(X_test)
print("Accuray:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(X_test,
    y_Pred))
```

```
O/P:-
Accuracy : 0.8182
Confusion Matrix:
[[6782  632]
 [1144  1211]]
Hyperparameter Tuning
best_acc = 0.
best_n = 0.
for n in [10,20,50,100,200]:
    model = AdaBoostClassifier(estimator = DecisionTree
        classifier(max_depth=1), n_estimators=n,
        random_state=42)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    print(f"n_estimator={n} -> Accuray = {acc}")
    if acc > best_acc:
        best_acc = acc
        best_n = n.
print("Best accuracy:", best_acc)
print("Best n_estimator:", best_n)
O/P:
Best accuracy = 0.8332
Best n_estimator = 200
```

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv("income.csv")

le = LabelEncoder()
for col in df.columns:
    if df[col].dtype == "object":
        df[col] = le.fit_transform(df[col])

X = df.iloc[:, :-1]
y = df.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

ada_default = AdaBoostClassifier(n_estimators=10, random_state=42)
ada_default.fit(X_train, y_train)
```

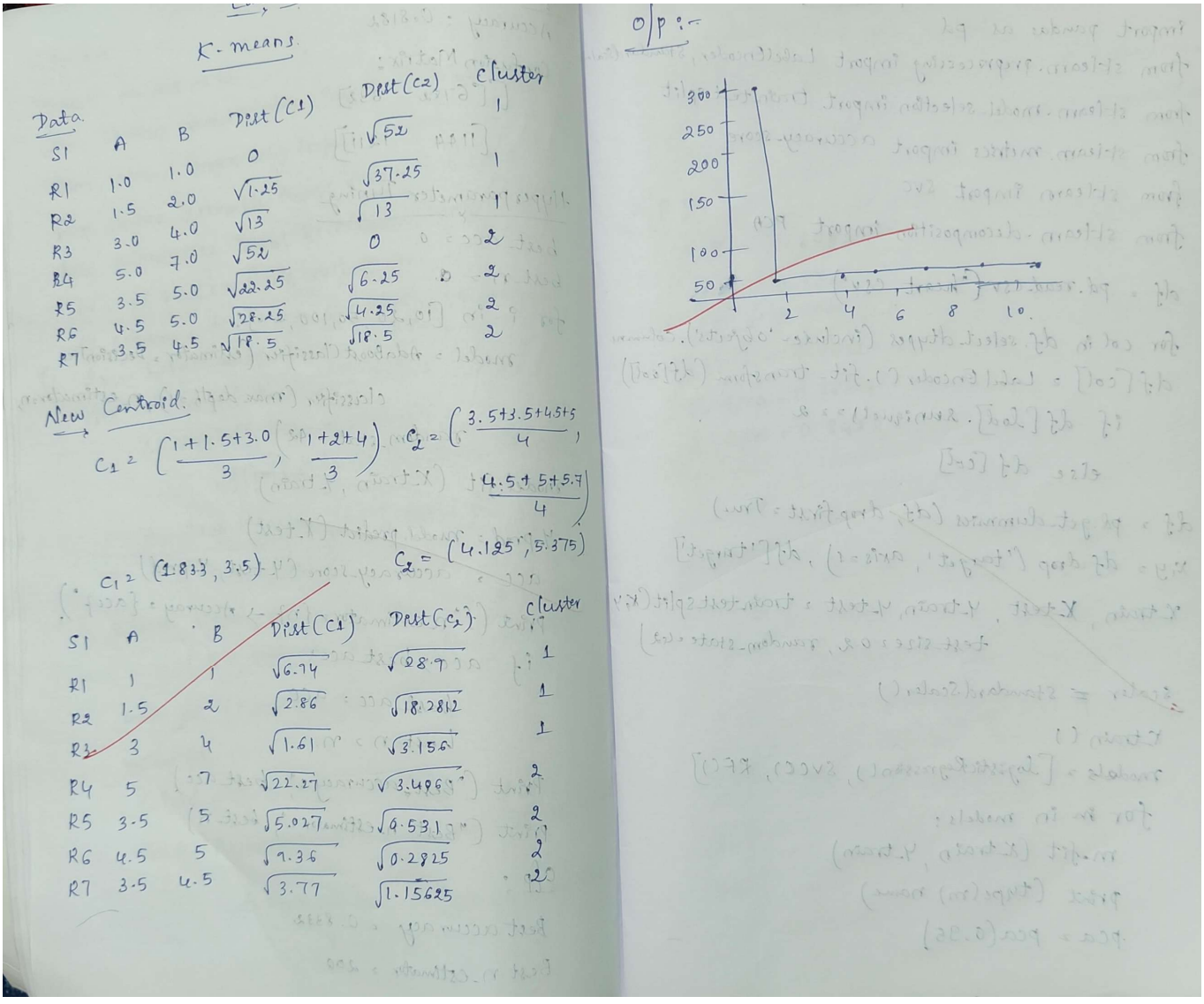
```
y_pred_default = ada_default.predict(X_test)
default_acc = accuracy_score(y_test, y_pred_default)

print("Default n_estimators = 10")
print("Accuracy:", default_acc)

tree_range = range(1, 101)
scores = []

for n in tree_range:
    ada = AdaBoostClassifier(n_estimators=n, random_state=42)
    ada.fit(X_train, y_train)
    y_pred = ada.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    scores.append(acc)

best_score = max(scores)
best_trees = tree_range[scores.index(best_score)]

print("\nBest Accuracy:", best_score)
print("Optimal number of trees:", best_trees)

plt.figure()
plt.plot(tree_range, scores)
```

```
plt.xlabel("Number of Trees")

plt.ylabel("Accuracy")

plt.title("AdaBoost Accuracy vs Number of Estimators")

plt.show()
```

Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file

Screenshot



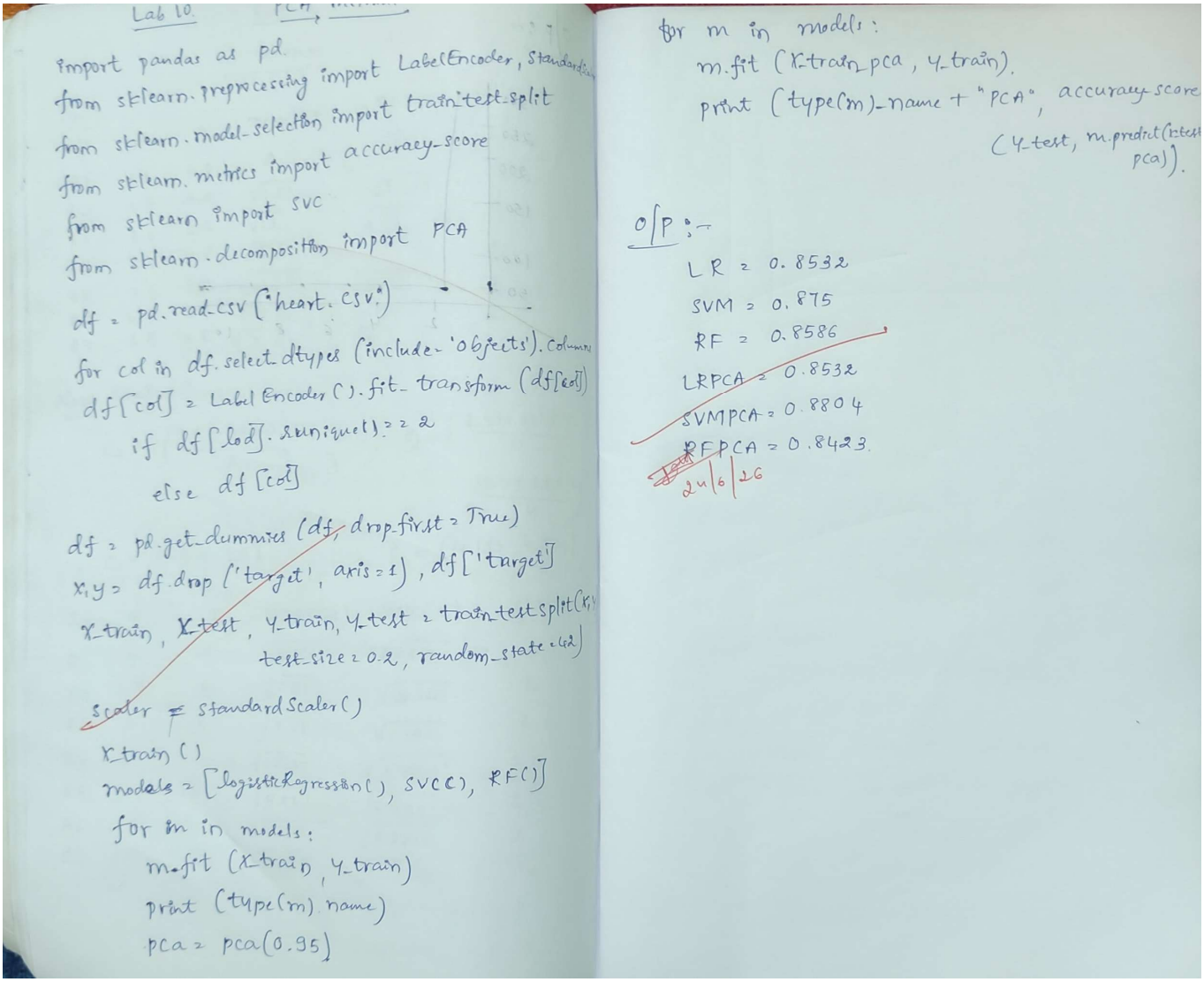
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("iris.csv")

df.head()

X = df[['petal_length', 'petal_width']]

X.head()

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_scaled[:5]

inertia = []
K_range = range(1, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
```

```
    inertia.append(kmeans.inertia_)

plt.figure()
plt.plot(K_range, inertia, marker='o')
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertia (WCSS)")
plt.title("Elbow Method")
plt.show()

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
y_pred = kmeans.fit_predict(X_scaled)

df['cluster'] = y_pred
df.head()

plt.figure()

plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_pred)
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], marker='X')

plt.xlabel("Petal Length (scaled)")
plt.ylabel("Petal Width (scaled)")
plt.title("K-Means Clustering (k=3)")
plt.show()
```

Program 11

Implement Dimensionality reduction using Principal Component Analysis (PCA) method.

Screenshot



Lab 10.

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn import svc
from sklearn.decomposition import PCA

df = pd.read_csv("heart.csv")
for col in df.select_dtypes(include='objects').columns:
    df[col] = LabelEncoder().fit_transform(df[col])
    if df[col].nunique() > 2
    else df[col]

df = pd.get_dummies(df, drop_first = True)
X, y = df.drop('target', axis=1), df['target']
X_train, X_test, y_train, y_test = train_test_split(X,y,
                    test_size = 0.2, random_state = 42)

scaler = StandardScaler()
X_train()
models = [LogisticRegression(), SVC(), RF()]
for m in models:
    m.fit(X_train, y_train)
    print(type(m).name)
    pca = pca(0.95)
```

```
for m in models:
    m.fit(X_train_pca, y_train).
    print(type(m)_name + "PCA", accuracy_score
                (y_test, m.predict(X_test
                pca)).
```

O/P :-

```
LR = 0.8532
SVM = 0.875
RF = 0.8586
LRPCA = 0.8532
SVMPCA = 0.8804
RFPCA = 0.8423
```

24/6/26

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.decomposition import PCA
from sklearn.metrics import accuracy_score

df = pd.read_csv("heart.csv")

X = df.iloc[:, :-1]
y = df.iloc[:, -1]

categorical_cols = X.select_dtypes(include=["object"]).columns.tolist()
numerical_cols = X.select_dtypes(exclude=["object"]).columns.tolist()

for col in categorical_cols:
    if X[col].nunique() == 2:
        le = LabelEncoder()
        X[col] = le.fit_transform(X[col])
```

```
categorical_cols = [col for col in categorical_cols if X[col].nunique() > 2]

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numerical_cols),
        ("cat", OneHotEncoder(drop="first"), categorical_cols)
    ]
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

models = {
    "SVM": SVC(),
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Random Forest": RandomForestClassifier()
}

print("=== Without PCA ===")
baseline_results = {}

for name, model in models.items():
```

```
    pipeline = Pipeline([
        ("preprocessing", preprocessor),
        ("classifier", model)
    ])

    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)
    acc = accuracy_score(y_test, y_pred)

    baseline_results[name] = acc
    print(f"{name}: {acc:.4f}")

print("\n=== With PCA ===")
pca_results = {}

for name, model in models.items():
    pipeline = Pipeline([
        ("preprocessing", preprocessor),
        ("pca", PCA(n_components=0.95)),
        ("classifier", model)
    ])

    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)
```

```
        acc = accuracy_score(y_test, y_pred)


        pca_results[name] = acc

        print(f"{name}: {acc:.4f}")


print("\n=== Comparison ===")

for name in models.keys():

        print(f"{name}: Without PCA = {baseline_results[name]:.4f}, With PCA =
{pca_results[name]:.4f}")
```